

Capitolo 11 - Memoria di Massa

Status Done

Memoria di massa (Mass Storage)

La memoria di massa è il sistema di **memorizzazione non volatile** di un computer.

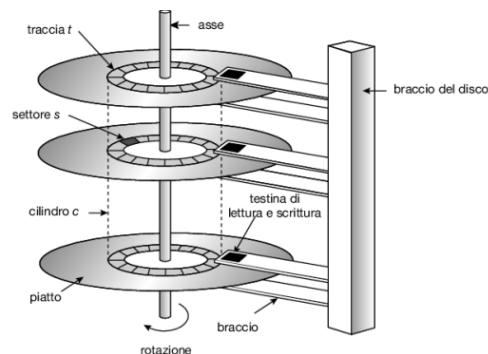
- **Memoria secondaria:** il principale sistema di archiviazione nei computer moderni. Fornita principalmente da **dischi rigidi (HDD)** e **dispositivi di memoria non volatile (NVM)**.
- **Memoria terziaria:** più lenta e più grande, composta tipicamente da nastri magnetici, dischi ottici o spazio di archiviazione su cloud.
- **Acronimo NVS (Non-Volatile Storage):** utilizzato per includere tutti i tipi di memoria di massa.

Struttura dei dispositivi di memorizzazione

I **dischi magnetici (HDD)** e i **dispositivi NVM** sono i supporti fondamentali della memoria secondaria. Il SO mappa le loro proprietà fisiche in **memoria logica** tramite la mappatura degli indirizzi.

Dischi rigidi (Hard Disks - HDD)

- **Struttura:** piatti (platters) rotondi rivestiti di materiale magnetico per registrare e leggere le informazioni.
- **Componenti:**
 - **Testine di lettura/scrittura (Read/Write Heads):** una per superficie, fissate al braccio del disco (disk arm).
 - **Tracce (tracks):** la superficie di un piatto è divisa logicamente in tracce circolari.
 - **Settori (sectors):** suddivisioni delle tracce, sono la più piccola unità di trasferimento (tradizionalmente 512 byte, ora spesso 4 KB).
 - **Cilindri (Cylinders):** l'insieme delle tracce corrispondenti alla stessa posizione del braccio.



- **Funzionamento e prestazioni:**
 - **Rotazione:** velocità elevata (es. 5400, 7200, 10000, 15000 RPM).
 - **Velocità di trasferimento (transfer rate):** velocità con cui i dati passano dal disco al computer.
 - **Tempo di posizionamento (accesso casuale):** tempo necessario per accedere a un dato, composto da:
 - **Tempo di ricerca (seek time):** tempo per spostare il braccio al cilindro desiderato.
 - **Latenza di rotazione (rotation latency):** tempo per far ruotare il settore desiderato sotto la testina.
- **Rischi:**
 - **Urto della testina (head crash):** la testina urta la superficie magnetica, causando danni irreparabili e perdita di dati.
- **Note:** i dischi possono essere rimovibili (hot-swappable) in alcuni sistemi.

Dispositivi NVM

I dispositivi NVM rappresentano un'evoluzione rispetto agli HDD perché sono basati su principi **elettrici** anziché meccanici. Nella maggior parte dei casi, sono costituiti da un **controllore** e da chip di memoria a semiconduttore **NAND Flash**.

Panoramica dei dispositivi NVM

I dispositivi NVM più comuni sono i **Dischi a Stato Solido (SSD)**.

- **Differenza fisica:** a differenza degli HDD che hanno piatti rotanti e bracci meccanici, gli SSD **non hanno parti in movimento**.
- **Vantaggi chiave:**
 - **Velocità superiore:** l'assenza di parti mobili elimina il *tempo di posizionamento* e la *latenza di rotazione*, rendendoli molto più rapidi degli HDD.
 - **Affidabilità:** sono meno soggetti a guasti meccanici (come l'urto della testina).
 - **Efficienza energetica:** consumano meno energia.
- **Svantaggi Storici:** tradizionalmente più costosi e con capacità inferiori rispetto ai dischi rigidi di fascia alta, anche se questo divario si sta riducendo rapidamente.
- **Prestazioni e Interfacce:** la loro elevata velocità può essere limitata dalle interfacce bus standard. Per superare questo collo di bottiglia, alcuni SSD si collegano direttamente alla bus di sistema, ad esempio tramite PCIe.

Sfide della tecnologia NAND Flash

La tecnologia NAND presenta limitazioni uniche che rendono la gestione dei dati complessa:

1. Operazioni asimmetriche:

- I dati possono essere letti e scritti a livello di pagina (analogo al settore).
- I dati **non possono essere sovrascritti** direttamente. Per aggiornare i dati, le celle devono essere prima cancellate.
- La **cancellazione** avviene a livello di **blocchi** (che contengono molte pagine) ed è molto più lenta della lettura e della scrittura.

2. Usura e Durata (Endurance):

- Le celle NAND si deteriorano progressivamente a ogni ciclo di cancellazione. Dopo un certo numero di cicli, non possono più conservare i dati in modo affidabile.
- La durata di un SSD è misurata in DDPD (Drive Writes per Day): quante volte è possibile scrivere la capacità totale dell'unità ogni giorno prima che si guasti.

A causa di queste complessità, il SO non gestisce direttamente le singole celle flash, ma comunica con il **controllore NVM** che si occupa di gestire tutti gli aspetti fisici.

Algoritmi del controllore delle NAND flash

Il controllore implementa algoritmi interni per superare le limitazioni del chip NAND, mantenendo le prestazioni e l'affidabilità.

1. Gestione dei dati non validi (garbage collection):

- Quando un blocco di dati viene aggiornato (riscritto), la versione precedente della pagina non viene cancellata immediatamente, ma viene marcata come non valida. Il nuovo dato viene scritto in una nuova pagina disponibile.
- Per tenere traccia di quali pagine sono valide, il controllore usa il **Flash Translation Layer (FTL)**.
- Quando non ci sono blocchi interamente liberi per le nuove scritture, si attiva la **garbage collection**: il controllore copia tutte le pagine valide da un blocco misto in un altro blocco libero. Una volta che il blocco di origine è completamente vuoto (contiene solo dati non validi), può essere cancellato e riutilizzato.

2. Over-Provisioning:

- Una percentuale della capacità totale del disco (spesso il 20%) viene riservata e non è visibile al SO.
- Questa area serve come spazio di lavoro cruciale per garantire che la garbage collection possa sempre trovare blocchi liberi per copiare i dati e ottimizzare le prestazioni di scrittura, soprattutto quando l'unità è quasi piena.

3. Livellamento dell'Usura (Wear Leveling):

- Per massimizzare la vita utile del dispositivo, il controllore utilizza algoritmi per distribuire equamente i cicli di cancellazione e scrittura su tutti i blocchi NAND.
- Se un blocco viene scritto e cancellato troppo spesso, l'algoritmo sposterà i dati su un blocco meno utilizzato, assicurando che tutti i blocchi si "consumino" alla stessa velocità.

4. Affidabilità e protezione:

- Gli NVM utilizzano codici di correzione degli errori (ECC) per rilevare e riparare piccoli errori che si verificano durante la lettura.
- In caso di guasti HW più gravi o irreversibili, la protezione RAID viene utilizzata per il recupero dei dati.

Memoria volatile

In questo contesto, la memoria volatile (DRAM) è discussa perché può essere utilizzata come dispositivo di archiviazione di massa ad altissima velocità, anche se i dati vengono persi in caso di interruzione di corrente.

Unità RAM (RAM Disks)

- **Funzionamento:** sono create da driver di periferica che prendono una sezione della DRAM di sistema e la presentano al SO come se fosse un'unità di memoria di massa (memoria secondaria).
- **Utilizzo:** possono essere usate come dispositivi a blocchi non formattati o, più comunemente, vi si creano **file system** per eseguire operazioni standard sui file.
- **Scopo:** consentono a utenti e programmatori di salvare temporaneamente i dati in memoria usando le normali operazioni sui file. Questo è utile per l'archiviazione temporanea ad alta velocità.
- **Vantaggio di velocità:** sebbene i dispositivi NVM siano veloci, la DRAM lo è molto di più. Le operazioni di I/O su unità RAM sono il modo più rapido per creare, leggere, scrivere ed eliminare file temporanei.
- **Esempi di utilizzo:** programmi che condividono dati scrivendo e leggendo su un'unità RAM; su Linux, la directory temporanea `/tmp` è spesso di tipo **"tmpfs"** (un'unità RAM). Linux crea anche un file system temporaneo all'avvio (initrd) prima di caricare i dispositivi di archiviazione principali.

Nastri magnetici:

- Sono stati i primi supporti di memorizzazione secondaria.
- **Caratteristiche:** hanno una capacità di memorizzazione enorme e permanente.
- **Limitazioni:** sono inadatti come memoria secondaria a causa del tempo di accesso estremamente elevato:
 - Sono dispositivi ad **accesso sequenziale** (il posizionamento sul dato può richiedere minuti).
 - Il tempo di accesso casuale è migliaia di volte maggiore rispetto ai dischi magnetici e circa centomila volte più lento degli SSD.
- **Usi principali:**
 1. Creazione di copie di backup dei dati.
 2. Registrazione di dati usati raramente.
 3. Trasferimento di informazioni tra sistemi diversi.
- **Prestazioni I/O:** una volta raggiunto il punto desiderato, la velocità di lettura/scrittura è paragonabile a quella di un'unità a disco. La capacità attuale è di diversi terabyte, spesso raddoppiata grazie alla compressione dati.

Metodi di connessione alla memoria secondaria

I dispositivi di memoria secondaria sono collegati al computer tramite il bus di sistema o un bus di I/O.

- **Tipi di bus comuni:** ATA, SATA (il più comune), eSATA, SAS, USB e Fibre Channel (FC).
- **Interfaccia NVM Express (NVMe):** un'interfaccia speciale e molto veloce creata per i dispositivi NVM, che li collega direttamente al bus PCI di sistema. Questo aumenta il throughput e riduce la latenza, superando i limiti di velocità delle interfacce HDD tradizionali.
- **Trasferimento dati:**
 - I trasferimenti sul bus sono gestiti da speciali processori chiamati **controllori** o **adattatori bus-host (HBA)**, situati sul lato computer del bus.
 - Ogni dispositivo di memoria di massa ha un proprio controllore di periferica integrato.
 - L'host inserisce il comando nell'HBA, che lo invia al controllore del dispositivo.
 - I controllori dei dispositivi hanno spesso una **cache integrata**. Il trasferimento dei dati tra il dispositivo e l'host avviene in genere a velocità elevate tramite DMA (Direct Memory Access), utilizzando la cache DRAM dell'host.

Mappatura degli indirizzi

I dispositivi di archiviazione sono gestiti dal SO come un grande vettore unidimensionale di blocchi logici.

- **Blocco logico:** è la più piccola unità di trasferimento.
- **Mappatura:** ogni blocco logico viene mappato su un settore fisico (HDD) o una pagina (NVM). L'intero vettore è mappato in sequenza sulle posizioni fisiche del dispositivo.
- **Indirizzo di blocco logico (LBA - Logical Block Address):** il SO utilizza un semplice LBA per indirizzare i dati, anziché tuple complesse (cilindro, traccia, settore).

Complessità della traduzione LBA → Indirizzo Fisico

Nonostante l'LBA sia logicamente sequenziale, la traduzione in indirizzi fisici "vecchio stile" (cilindro, traccia, settore) è difficile per tre motivi:

1. **Settori difettosi:** il disco nasconde i settori danneggiati, sostituendoli con settori di riserva in altre posizioni. L'LBA rimane sequenziale, ma la posizione fisica è alterata.
2. **Numero variabile di settori per traccia:** sui dischi moderni, il numero di settori per traccia non è costante per massimizzare la densità.
3. **Gestione interna:** i produttori gestiscono la mappatura LBA/fisica internamente, limitando la correlazione diretta tra LBA e posizione fisica esterna.

Il concetto di CLV e CAV serve a illustrare il **secondo motivo** per cui la mappatura da **LBA (Indirizzo di Blocco Logico)** a indirizzo fisico è complessa:

Sui dischi, la circonferenza delle tracce aumenta man mano che ci si allontana dal centro. Per sfruttare al massimo lo spazio disponibile sulle tracce più esterne (più lunghe), è necessario memorizzare più settori su di esse. Questo significa che il **numero di settori per traccia non è costante** su tutto il disco.

Per gestire questa variazione di densità, esistono due approcci principali legati al modo in cui il disco ruota e legge i dati:

- **Velocità Lineare Costante (CLV - Constant Linear Velocity):**
 - Densità di bit uniforme.
 - Le tracce esterne hanno più settori (lunghezza maggiore).
 - L'unità aumenta la velocità di rotazione quando la testina si sposta dalle tracce esterne a quelle interne per mantenere costante la quantità di dati che scorre sotto la testina (usato in CD/DVD-ROM).
- **Velocità Angolare Costante (CAV - Constant Angular Velocity):**
 - La velocità di rotazione del disco **rimane costante**.
 - La densità dei bit diminuisce spostandosi dall'interno all'esterno per mantenere costante la velocità di trasmissione dati.
 - Questo è il metodo utilizzato nei dischi rigidi (HDD).

Scheduling dei dischi rigidi

Una responsabilità fondamentale del SO è garantire un uso efficiente dell'HW, che per i dischi rigidi significa assicurare **tempi di accesso contenuti e ampiezze di banda (bandwidth) elevate**.

- **Componenti del tempo di accesso (per dispositivi meccanici):**
 1. **Tempo di ricerca (seek time):** tempo necessario per spostare le testine al cilindro desiderato.
 2. **Latenza di rotazione (Rotational Latency):** tempo aggiuntivo per attendere che il settore desiderato si trovi sotto la testina.
- **Ampiezza di banda (Bandwidth) del Disco:** numero totale di byte trasferiti diviso il tempo totale trascorso.
- **Obiettivo dello scheduling:** migliorare sia il tempo di accesso che l'ampiezza di banda gestendo l'ordine delle richieste I/O del disco.

Richieste I/O

Quando un processo necessita di I/O, effettua una chiamata di sistema che specifica:

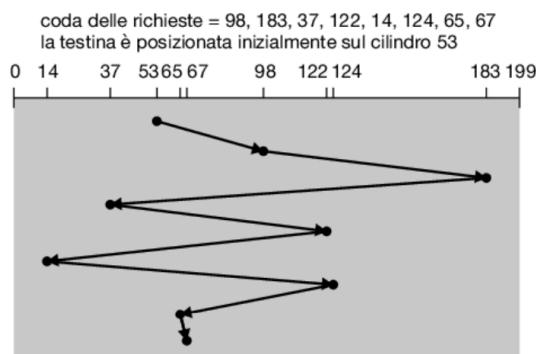
- a. tipo di operazione (input/output);
- b. indirizzo sul disco (LBA);
- c. indirizzo di memoria per il trasferimento;
- d. numero di settori da trasferire.

- **Coda di richieste:** se il disco è occupato, le richieste vengono aggiunte a una coda. Il driver del dispositivo può migliorare le prestazioni riordinando opportunamente questa coda per minimizzare i tempi di ricerca.
- **Contesto moderno:** le unità disco attuali gestiscono internamente la mappatura LBA/indirizzo fisico, sollevando l'host da questo controllo.
- **Obiettivi attuali:** equità, tempestività e ottimizzazione (es. raggruppamento di I/O sequenziali).
- **Assunzione per algoritmi:** sebbene la posizione fisica non sia nota con precisione, gli algoritmi possono assumere che un incremento dell'LBA implichi un incremento degli indirizzi fisici e che LBA vicini mappino a blocchi fisici vicini.

Scheduling in ordine d'arrivo - FCFS (First Come, First Served)

L'algoritmo più semplice: le richieste sono servite nell'ordine in cui arrivano.

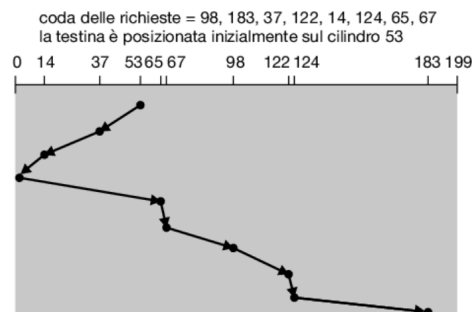
- **Caratteristiche:** è intrinsecamente **equo**.
- **Limiti:** non garantisce la massima velocità del servizio. Può causare un movimento della testina inefficiente e dispendioso, come evidenziato dai grandi salti non necessari.
- **Esempio:** se la testina è al cilindro 53 e la coda è 98, 183, 37, 122, 14, 124, 65, 67, il movimento totale è di 640 cilindri.



Scheduling - SCAN

L'algoritmo SCAN (detto anche algoritmo dell'ascensore) muove il braccio del disco in una sola direzione, servendo tutte le richieste che incontra, fino a raggiungere l'estremo. A quel punto, inverte la marcia e ripete il processo nell'altra direzione.

- **Funzionamento:** la testina attraversa continuamente il disco in entrambe le direzioni.
- **Vantaggi:** garantisce che le richieste per i cilindri vicini vengano servite insieme, riducendo il tempo di ricerca totale rispetto a FCFS.
- **Limiti:** le richieste appena superate dalla testina devono attendere che il braccio raggiunga l'estremo opposto e inverta il movimento, portando a tempi di attesa maggiori e più variabili per i cilindri vicini agli estremi raggiunti di recente.
- **Esempio:** Se il movimento iniziale è verso 0 (partendo da 53), serve: 37, 14, 0 (inversione), e poi 65, 67, 98, 122, 124, 183.

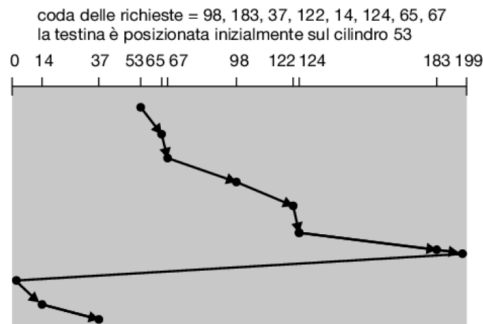


Scheduling - C-SCAN (Circular SCAN)

È una variante di SCAN progettata per garantire un **tempo d'attesa meno variabile**.

- **Funzionamento:** la testina si muove da un estremo all'altro, servendo le richieste lungo il percorso **in una sola direzione**.

- **Viaggio di ritorno:** quando la testina raggiunge l'estremo opposto, **ritorna immediatamente (salta)** all'inizio del disco **senza servire alcuna richiesta** durante il viaggio di ritorno.
- **Concetto:** tratta il disco come una **lista circolare** (il primo e l'ultimo cilindro sono adiacenti logicamente).
- **Vantaggi:** le richieste non devono attendere l'intero viaggio di andata e ritorno per essere servite, riducendo la variabilità del tempo di attesa.
- **Esempio:** se la direzione è da 0 a 199 (partendo da 53), serve: 65, 67, 98, 122, 124, 183. Raggiunge l'estremo (199) e salta immediatamente a 0 per ricominciare.



Scelta di un algoritmo di scheduling

La scelta dell'algoritmo dipende dalle condizioni operative e dalle esigenze del sistema.

- **Sistemi scarichi:** se la coda ha solo una richiesta, tutti gli algoritmi si comportano come FCFS.
- **Sistemi sotto sforzo:** SCAN e C-SCAN offrono prestazioni migliori perché riducono il rischio di *attesa indefinita* (starvation), un problema potenziale in ambienti multi-programmazione.

Scheduler in Linux

Linux implementa diversi scheduler per affrontare le sfide dell'I/O:

Scheduler	Principi e Funzionamento	Utilizzo Tipico
Scheduler con Scadenza (Deadline Scheduler)	Mantiene code di lettura e scrittura separate. Dà priorità alla lettura (i processi si bloccano di più in lettura). Le richieste vengono ordinate per LBA (implementando C-SCAN). Ha anche code FCFS separate per garantire che nessuna richiesta superi un tempo di attesa preconfigurato (es. 500 ms).	Scheduler predefinito in distribuzioni come RedHat 7.
NOOP (No Operation)	Implementazione minimalista. Ideale per sistemi CPU-bound o che utilizzano dispositivi I/O veloci (come gli NVM/SSD), dove lo scheduling complesso non è necessario o addirittura dannoso.	Consigliato su sistemi con NVM/SSD.
CFQ (Completely Fair Queuing)	Mantiene tre code separate con priorità decrescente (Real Time, Best Effort, Idle). Utilizza l' insertion sort per ordinare le code per LBA. Cerca di prevedere se un processo genererà presto ulteriori richieste di I/O e attende, minimizzando i movimenti della testina assumendo la <i>località di riferimento</i> delle richieste di un singolo processo.	Predefinito per le unità SATA.

Scheduling di dispositivi NVM

A differenza degli HDD (dischi meccanici) dove lo scheduling mira a ridurre il movimento della testina, nei dispositivi NVM (che non hanno parti in movimento) l'approccio è più semplice.

- **Politica di base:** generalmente si adotta una semplice politica FCFS (First Come, First Served), spesso modificata (come nello scheduler Noop di Linux) per **fondere richieste adiacenti**.
- **Scheduling specifico per NVM:**
 - Il tempo per le letture è uniforme.
 - Il tempo per le scritture non è uniforme a causa delle proprietà della memoria flash (necessità di cancellazione, garbage collection).
 - Alcuni scheduler fondono solo le richieste di scrittura adiacenti, mentre le richieste di lettura sono servite in ordine FCFS.

Confronto prestazioni: HDD vs. NVM

Tipo di I/O	HDD (Meccanico)	NVM (SSD)	Vantaggio NVM
-------------	-----------------	-----------	---------------

Sequenziale	Ottimale (tempo di ricerca minimo).	Prestazioni migliori (fino a un ordine di grandezza superiore).	Inferiore, il guadagno è limitato dalla velocità di trasferimento dati.
Casuale (Random)	Richiede movimento della testina, molto lento (poche centinaia di IOPS).	Molto veloce, l'assenza di testina è un enorme vantaggio (centinaia di migliaia di IOPS).	Massiccio, è il punto di forza degli SSD.

Variazioni nelle prestazioni di scrittura NVM

Le prestazioni di scrittura negli SSD sono meno costanti rispetto agli HDD:

- **Scrittura vs. Lettura:** la scrittura su NVM è più lenta della lettura, riducendo il vantaggio sul sequenziale rispetto agli HDD.
- **Fattori di decadimento:** le prestazioni di scrittura variano in base a:
 1. **Quanto è pieno il dispositivo:** più è pieno, maggiore è la necessità di *garbage collection*.
 2. **Usura (cicli di cancellazione):** un dispositivo usurato o vicino al termine del ciclo di vita ha prestazioni notevolmente inferiori.

Amplificazione di scrittura (Write Amplification)

La necessità di gestire i dati non validi e l'usura genera operazioni di I/O aggiuntive non richieste direttamente dall'applicazione.

- **Funzionamento:** una singola richiesta di scrittura da parte dell'applicazione può innescare una **Garbage Collection** per liberare spazio. Questo processo può causare:
 1. Una scrittura di pagina (i dati nuovi).
 2. Una o più letture di pagine (dei dati validi dal blocco da raccogliere).
 3. Una o più scritture di pagine (dei dati validi riscritti nell'area di over-provisioning).
- **Impatto:** questa attività interna (non proveniente dall'applicazione) è detta amplificazione di scrittura e può avere un impatto significativo, aggiungendo diverse operazioni I/O per ogni singola richiesta di scrittura.

Rilevamento e correzione di errori

Il rilevamento e la correzione degli errori sono meccanismi cruciali per garantire l'integrità dei dati in tutti i sistemi di memoria.

Rilevamento degli errori (Error Detection)

Consente al sistema di identificare se un bit è stato modificato in modo involontario, evitando la propagazione di dati corrotti.

- **Bit di parità:** metodo semplice di rilevamento:
 - Viene aggiunto un bit per ogni byte (o parola).
 - Registra se il numero di bit "1" nel byte è pari (parità = 0) o dispari (parità = 1).
 - Se un singolo bit viene corrotto, la parità memorizzata non corrisponde più a quella calcolata, segnalando l'errore.
 - **Limite:** non è in grado di rilevare errori su due bit contemporaneamente.
- **Checksum:** la parità è una forma di checksum (usa l'aritmetica modulare).
- **Controllo di ridondanza ciclica (CRC):** un metodo più robusto, comune nelle reti, che utilizza una funzione hash per rilevare errori su più bit.

Codici di correzione degli errori (ECC - Error-Correcting Code)

L'ECC non solo rileva un errore, ma è anche in grado di correggerlo.

- **Funzionamento:** l'ECC è un valore calcolato in base ai dati di un settore (HDD) o di una pagina (NVM) e viene archiviato insieme ai dati.
- **Correzione:** quando i dati vengono letti, l'ECC viene ricalcolato e confrontato con il valore memorizzato. Se differiscono, ma solo per pochi bit, l'ECC contiene abbastanza informazioni ridondanti per identificare quali bit sono stati modificati e calcolare i loro valori corretti.
- **Classificazione errori:**
 - **Errore reversibile (Soft Error):** l'ECC riesce a correggere l'errore.
 - **Errore irreversibile (Hard Error):** ci sono troppe modifiche e l'ECC non può correggere l'errore.
- **Implementazione:** il controllore del disco o dell'unità flash gestisce automaticamente il calcolo e la verifica dell'ECC durante ogni operazione di lettura e scrittura.
- **Importanza:** l'uso dell'ECC è un fattore che spesso distingue i prodotti HW di livello aziendale da quelli di consumo.

Gestione delle unità di memoria secondaria

Il SO è responsabile dell'inizializzazione del dispositivo, dell'avviamento del sistema e della gestione dei blocchi difettosi.

Formattazione del disco, partizioni e volumi

Prima che un dispositivo di storage possa memorizzare dati, deve essere inizializzato.

Formattazione di basso livello (Fisica)

- **Processo:** divisione del supporto in settori (HDD) o inizializzazione delle pagine e creazione della FTL (NVM).
- **Struttura:** ogni locazione riceve una speciale struttura dati (intestazione, area dati, coda) contenente informazioni del controllore, come il numero di settore e il codice di correzione degli errori (ECC).
- **Responsabilità:** viene eseguita prevalentemente dal costruttore per testare il dispositivo e inizializzare la mappatura tra blocchi logici e settori/pagine funzionanti.

Strutture dati del SO

Dopo la formattazione fisica, il SO esegue tre passi per rendere il dispositivo utilizzabile per i file:

1. **Partizionamento:**

- Il dispositivo viene suddiviso in uno o più gruppi di blocchi/pagine chiamati **partizioni**.
- Ogni partizione può essere trattata come un'unità separata (es. una per il file system, una per lo swap).
- Le informazioni sulle partizioni sono scritte in una posizione fissa (es. il comando `fdisk` di Linux gestisce questo).

2. **Creazione e gestione del volume:**

- A volte un file system è inserito direttamente in una singola partizione.
- Altre volte, un **Volume Manager** (es. `lvm2` di Linux, o ZFS) gestisce più partizioni o dispositivi insieme, distribuendo uno o più file system su di essi (es. RAID). Un "volume" è qualsiasi file system montabile.

3. **Formattazione logica (creazione del file system):**

- Il SO registra le strutture dati iniziali del file system (es. descrizioni dello spazio libero e allocato, directory radice vuota).

File system e blocchi

- **Avvio:** le informazioni sulle partizioni indicano quale contiene il file system avviabile (**root**). In Windows, i volumi sono separati da lettere (C:, D:); in Linux, i volumi sono montati all'interno della stessa struttura ad albero.
- **Cluster:** per maggiore efficienza, la maggior parte dei file system raggruppa i blocchi in cluster. L'I/O del file system procede per cluster, favorendo l'accesso sequenziale su HDD.
- **Disco di basso livello (Raw Disk):** una partizione del disco usata come un grande array sequenziale di blocchi senza alcuna struttura di file system.
 - **I/O di basso livello (Raw I/O):** bypassa tutti i servizi del file system (cache, locking, allocazione, nomi di file).
 - **Usi:** spazio di swap; sistemi di gestione di database che vogliono un controllo preciso sulla posizione dei record.

Blocco d'avviamento (Bootstrap Block)

Per avviarsi, un computer esegue un programma iniziale (**bootstrap**) memorizzato nel **firmware** (NVM) sulla scheda madre.

- **Bootstrap loader:** un piccolo programma nel firmware che è in grado di caricare un programma di *bootstrap completo* dalla memoria secondaria, memorizzato nei **blocchi di avvio** (o **settore d'avvio**) in una posizione fissa del dispositivo.
- **Disco/Partizione di avvio:** un dispositivo o una partizione contenente il codice necessario all'avvio.
- **Processo d'avviamento (Windows Esempio):**
 1. Il codice nel firmware legge il codice d'avviamento dal **Master Boot Record (MBR)** (primo blocco logico/pagina del disco).
 2. L'MBR contiene il codice d'avviamento e una tabella che elenca le partizioni e indica quella da cui avviare il sistema.
 3. Il sistema legge il primo settore di quella partizione (settore d'avviamento) ed esegue le procedure finali di caricamento del SO.

Blocchi difettosi (Bad Block)

I malfunzionamenti di uno o più settori (o pagine) sono comuni, anche nei dischi nuovi. La gestione varia in base al controllore.

Gestione manuale (dischi semplici, es. IDE)

- **Procedura:** si esegue una scansione del disco (es. comando `badlocks` di Linux) per rilevare i blocchi difettosi.
- **Azione:** il blocco difettoso viene marcato come inutilizzabile per evitare che le procedure di allocazione del file system lo usino. I dati persi devono essere recuperati manualmente (es. da backup)

Strategie con controllore avanzato (HDD)

I controllori più complessi usano settori di riserva (**spare sectors**) nascosti al SO e gestiti internamente.

- **Accantonamento di settori (Sector Sparing / Sector Forwarding):**
 - Il controllore mantiene una lista dei blocchi malfunzionanti.
 - Quando un settore difettoso viene rilevato, il controllore lo sostituisce logicamente con un settore di riserva inutilizzato.
 - Ogni richiesta per il blocco logico difettoso viene tradotta dal controllore nell'indirizzo del settore di riserva.
 - Per non compromettere lo scheduling I/O, i dischi mantengono settori di riserva nello stesso cilindro del blocco difettoso, quando possibile.
- **Traslazione dei settori (Sector Slipping):**
 - Se un blocco diventa difettoso, i settori successivi a quello difettoso vengono spostati in avanti di una posizione.
 - Questo libera uno spazio sequenziale sul disco che viene utilizzato come blocco sostitutivo, mantenendo la sequenzialità logica.
- **Errori irreversibili:** un errore irreversibile causa una perdita di dati e richiede un intervento manuale di ripristino.

Blocchi difettosi NVM

La gestione dei difetti sui dispositivi NVM è più semplice perché non influisce sul tempo di ricerca.

- **Gestione:** il controllore NVM alloca pagine di riserva o utilizza spazio dall'area di *over-provisioning*.
- **Azione:** il controllore mantiene una tabella delle pagine non valide e le esclude da qualsiasi operazioni di scrittura o lettura.

Gestione dell'area d'avvicendamento (Swapping)

L'avvicendamento (swapping) è l'operazione di spostamento dei dati tra il disco e la memoria centrale (RAM).

- **Evoluzione del concetto:** inizialmente, lo swapping spostava interi processi. Oggi, è combinato con le tecniche di memoria virtuale (paginazione) e sposta quasi sempre solo pagine di memoria. Per questo, *avvicendamento* e *paginazione* sono spesso usati come termini intercambiabili.
- **Ruolo del disco:** la memoria virtuale utilizza lo spazio su disco (l'area di swapping) come un'estensione, sebbene molto più lenta, della memoria centrale.
- **Obiettivo di progettazione:** fornire il miglior throughput possibile per il sistema di memoria virtuale, poiché l'uso dell'area di swapping riduce drasticamente le prestazioni.

Uso dell'area di avvicendamento

L'uso e la dimensione dell'area di swapping variano a seconda del SO e degli algoritmi di gestione della memoria:

- **Sistemi a pagina:** memorizzano solo le pagine non presenti nella memoria centrale (pagine inattive).
- **Dimensione:** varia da pochi megabyte ad alcuni gigabyte. È più prudente sovrastimare la dimensione poiché esaurire l'area può portare all'arresto o alla terminazione forzata dei processi.
- **Esempi di stima:**
 - Solaris: raccomanda uno spazio pari alla memoria virtuale che eccede la memoria fisica paginabile.
 - Linux: un tempo suggeriva il doppio della memoria fisica, ora è minore.
- **Area multipla:** sistemi come Linux consentono l'uso di aree di avvicendamento multiple (sia file che partizioni dedicate), poste su unità a disco distinte per distribuire il carico I/O della paginazione e migliorare il throughput.

Collocazione dell'area d'avvicendamento

L'area di swapping può essere collocata in due modi principali:

1. **All'interno del file system (Swap File):**
 - È un grande file creato e gestito attraverso le normali funzioni del file system.
 - **Vantaggio:** facile da creare, nominare e gestire.
2. **In una partizione dedicata (Raw Partition):**

- È una partizione del disco separata e non formattata (non contiene strutture di file system/directory).
- **Gestione:** utilizza un gestore dell'area d'avvicendamento ottimizzato per la velocità di accesso (non per lo spazio, come avviene nel file system).
- **Vantaggi:** migliori prestazioni grazie a algoritmi ottimizzati. La frammentazione non è un problema duraturo perché l'area viene reinizializzata all'avvio.
- **Svantaggi:** richiede l'assegnazione di una dimensione fissa al momento del partizionamento del disco e cambiarla richiede un ripartizionamento complesso.
- **Strategia mista:** sistemi come Linux permettono all'amministratore di scegliere tra entrambe le soluzioni, bilanciando la facilità di gestione (swap file) contro le migliori prestazioni (raw partition).

Connessione dei dispositivi di memorizzazione

I computer accedono alla memoria secondaria in tre modi principali: collegata direttamente alla macchina (host-attached), collegata tramite una rete locale (network attached) o tramite Internet (cloud).

Memoria secondaria connessa alla macchina (Host-Attached Storage)

Si accede a questa memoria tramite le porte I/O locali della macchina (host).

- **Tecnologie comuni:** la più comune è **SATA**. Per i sistemi di fascia alta si usano architetture più sofisticate come **Fibre Channel** (FC), che offre alta velocità e flessibilità grazie alla sua natura commutata, consentendo di connettere più macchine e dispositivi di memorizzazione.
- **Dispositivi:** include unità a disco, NVM, unità ottiche (CD/DVD/Blu Ray) e nastri magnetici.
- **Trasferimenti:** i comandi di I/O avviano trasferimenti tramite letture e scritture di blocchi logici diretti e unità specificamente identificate.

Memoria secondaria connessa alla rete (NAS - Network-Attached Storage)

Un dispositivo NAS fornisce accesso allo spazio di archiviazione attraverso una rete locale (LAN).

- **Architettura:** il NAS è spesso un sistema di memoria specializzato (storage array) o un computer che condivide il suo spazio con altri host sulla rete.
- **Protocolli di accesso:**
 - **NFS (Unix/Linux) e CIFS (Windows):** i client accedono alla memoria tramite un'interfaccia **RPC** (Remote Procedure Call), trattando il NAS come un file system remoto.
 - **iSCSI:** un protocollo più recente che usa la rete IP per trasportare il protocollo SCSI. Invia blocchi logici (non parti di file) sulla rete, permettendo al client di trattare lo storage come se fosse collegato direttamente alla macchina.
- **Vantaggi:** facile condivisione e gestione dello spazio di storage tra tutti i computer in una LAN.
- **Svantaggi:** tendenzialmente meno efficiente e con prestazioni inferiori rispetto ai sistemi host-attached, poiché compete con il normale traffico di rete.

Memoria secondaria su cloud (Cloud Storage)

Fornisce accesso allo spazio di archiviazione attraverso **Internet** o una **WAN** verso un data center remoto (servizio a pagamento).

- **Differenza NAS vs. Cloud:**
 - **Accesso NAS (CIFS/NFS):** l'accesso è come a un normale file system, integrato dal SO.
 - **Accesso Cloud:** basato su API. I programmi accedono allo storage utilizzando queste API specifiche (es. Amazon S3, Dropbox, Microsoft OneDrive, Apple iCloud).
- **Motivo dell'uso delle API:** gestire meglio la latenza elevata e i potenziali scenari di errore di una WAN (Internet). I protocolli NAS sono ottimizzati per le LAN (latenza bassa). In caso di interruzione, un'applicazione che usa il cloud storage si interrompe semplicemente, anziché bloccarsi come potrebbe fare un sistema NFS o CIFS.

Storage-Area Network (SAN) e Storage Array

Queste architetture sono progettate per sistemi di grandi dimensioni e server, separando il traffico di archiviazione del traffico dati generale della rete.

Storage-Area Network (SAN)

- **Definizione:** è una rete privata dedicata tra i server e le unità di memoria secondaria, che utilizza protocolli specifici per l'archiviazione (come Fibre Channel o iSCSI).
- **Vantaggi:**
 - **Isolamento:** le operazioni di I/O di storage non impegnano la banda della rete dati principale (LAN), riducendo la latenza generale.
 - **Flessibilità:** permette di connettere molte macchine e *storage array*. Lo spazio può essere allocato dinamicamente agli host.
 - **Condivisione:** consente ai cluster di server di condividere lo stesso storage.
- **Componenti:** uno **switch SAN** gestisce e controlla l'accesso degli host allo spazio di archiviazione.
- **Connettività:** avviene a breve distanza e in genere senza routing (a differenza del NAS).

Storage Array

- **Definizione:** un dispositivo costruito appositamente per la memorizzazione che contiene le unità fisiche (dischi o SSD) e un controllore intelligente (o più controllori ridondanti).
- **Funzionalità del controllore:** esegue una vasta gamma di funzioni di gestione come la protezione RAID, snapshot, replica, compressione, deduplicazione e crittografia.
- **Media di archiviazione:** possono essere composti solo da SSD (massime prestazioni) o da un mix di SSD e HDD, utilizzando gli SSD come cache per i dati più richiesti.
- **Connessione:** includono porte SAN (es. FC) e/o porte di rete (es. iSCSI).

Strutture RAID (Redundant Array of Independent Disks)

Le strutture RAID combinano più unità di memorizzazione (dischi o NVM) in unico array logico per migliorare due aspetti: **prestazioni** (parallelismo) e **affidabilità** (ridondanza).

- **Significato di RAID:** originariamente Redundant Array of Inexpensive Disks, oggi la "I" sta per **Independent**, sottolineando che l'obiettivo primario non è più il costo, ma l'affidabilità e la velocità di trasferimento.

Miglioramento dell'affidabilità tramite la ridondanza

La probabilità di un guasto aumenta con il numero di dischi. Ad esempio, una batteria di 100 dischi, con un MTBF (Mean Time Between Failures) del disco singolo di 100000 ore, avrà un MTBF dell'array di sole 1000 ore. La ridondanza è essenziale per evitare la perdita frequente di dati.

- **Mirroring (Duplicazione):** il metodo di ridondanza più semplice e costoso (**RAID Livello 1**). Ogni disco logico è composto da due dischi fisici. Ogni scrittura avviene su entrambi.
 - **Affidabilità:** i dati si perdono solo se il secondo disco si guasta prima che il primo guasto venga riparato. L'MTBF di un sistema mirrored è significativamente superiore (es. 57000 anni nell'esempio dato).
 - **Coerenza (caduta di tensione):** per evitare che i dischi mirrorati diventino incoerenti in caso interruzione di corrente, si possono usare memorie NVRAM (Non-Volatile RAM) protette (come cache) che garantiscono il completamento della scrittura dei blocchi.

Miglioramento delle prestazioni tramite il parallelismo

L'uso di più unità in parallelo migliora la capacità di gestire più richieste e/o la velocità di trasferimento (throughput).

- **Parallelismo con Mirroring (Lettura):** la frequenza delle richieste di lettura gestibili raddoppia, perché ogni richiesta può essere indirizzata indifferentemente a uno dei due dischi.
- **Striping (Sezionamento):** distribuzione dei dati in sezioni su più dispositivi per migliorare la capacità di trasferimento.
 - **Striping a livello di bit:** i bit di ciascun byte sono distribuiti su più dischi (es. 8 dischi per 8 bit). L'array agisce come un unico disco con una capacità di trasferimento otto volte superiore. Ogni accesso coinvolge tutti i dischi.
 - **Striping a livello di blocco:** il più comune. I blocchi di un file sono distribuiti in sequenza sui vari dispositivi.

▼ Cos'è lo striping?

È una tecnica che consiste nel suddividere i dati in piccole sezioni contigue e distribuire queste sezioni su più dischi separati all'interno di un array.

Immagina di dover copiare un file molto grande.

- **Senza Striping (Disco Singolo):** Il file viene letto e scritto sequenzialmente da un'unica unità, limitato dalla velocità massima di quel singolo disco.
- **Con Striping (Array di N Dischi):** Il file viene diviso in N pezzi (stripes) e questi pezzi vengono letti o scritti **contemporaneamente** da tutti gli N dischi.

Risultato: Si accede al file in parallelo, aumentando significativamente la velocità di trasferimento dei dati (throughput) quasi di un fattore N.

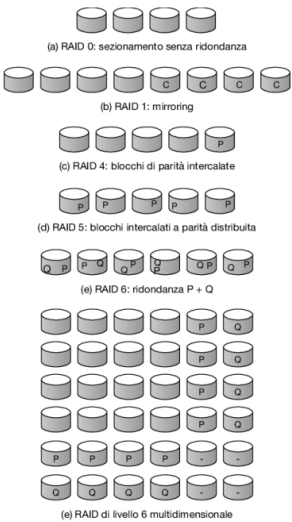
Lo striping permette di trattare un insieme di dischi come un'unica unità super-veloce che non è limitata dalla velocità di un singolo componente.

- **Obiettivi dello striping:** aumentare il throughput per accessi multipli a piccoli dati e ridurre il tempo di risposta per accessi a grandi quantità di dati.

Livelli RAID

Sono schemi che combinano striping e ridondanza (mirroring o parità) per bilanciare costi e prestazioni.

Livello RAID	Descrizione	Ridondanza	Vantaggi/Note
Livello 0	Striping a livello di blocchi senza ridondanza .	Nessuna (solo dati D)	Massime prestazioni di I/O. Nessuna protezione contro i guasti.
Livello 1	Mirroring (Duplicazione).	Completa (Dati D duplicati C)	Alta affidabilità, ripristino rapidissimo. Costoso (50% di overhead).
Livello 4	Striping con Parità Dedicata (ECC - Error Correcting Code).	Parità in un disco dedicato (P)	Efficienza di spazio superiore al RAID 1. Ottimo throughput in lettura parallela.
Ciclo Lettura-Modifica-Scrittura (RAID 4):	Una singola scrittura richiede 4 accessi (leggere il vecchio dato, leggere la vecchia parità, calcolare la nuova parità, scrivere il nuovo dato e la nuova parità). Questo rallenta le scritture piccole.		
Livello 5	Striping con Parità Distribuita .	Parità P distribuita su tutti i dischi.	Il più comune dei RAID basati su parità. Evita il "collo di bottiglia" dell'unità di parità del RAID 4. Buona protezione e spazio efficiente.
Livello 6 (P + Q)	Simile al RAID 5, ma memorizza due blocchi di ridondanza (P e Q) usando la matematica dei campi di Galois.	Doppia parità (P e Q)	Può tollerare il guasto simultaneo di due dischi .
Livello 0 + 1	Striping su un set di dischi, poi l'intero set è mirrorato .	Mirroring + Striping	Ottime prestazioni e affidabilità. Molto costoso (50% di overhead).
Livello 1 + 0	Mirroring delle coppie di dischi, poi striping delle coppie.	Mirroring + Striping	Prestazioni simili al 0+1. Più resiliente al guasto di un singolo disco rispetto al 0+1, perché un guasto non rende inaccessibile un'intera sezione.



Implementazione RAID

Il RAID può essere implementato a diversi livelli nell'architettura di sistema:

- **Software:** implementato nel kernel o nei programmi di sistema (es. volume manager). Soluzione flessibile.
- **Hardware (HBA):** implementato nell'adattatore bus-host. Soluzione economica ma poco flessibile.
- **Hardware (Storage Array):** implementato nel controllore dell'array. Offre la massima funzionalità (RAID, snapshot, replica) e può essere condiviso tra più host (SAN).
- **Virtualizzazione SAN:** implementato da un dispositivo intermediario che gestisce l'accesso allo storage.

Funzionalità aggiuntive:

- **Snapshot (Istantanea):** immagine del file system in un momento specifico.
- **Replica:** duplicazione automatica di scritture su siti remoti (sincrona o asincrona) per il *disaster recovery*.
- **Hot spare (disco di scorta):** un disco inutilizzato che subentra automaticamente a un disco guasto per ripristinare l'integrità del RAID senza intervento umano.

Scelta di un livello di RAID

La scelta dipende dalle esigenze di prestazioni, affidabilità e costi. Una considerazione chiave è il **tempo di ricostruzione**.

- **RAID 0:** usato in applicazioni ad alte prestazioni dove la perdita di dati non è critica (es. calcolo scientifico, dove i dati sono ricaricabili).
- **RAID 1:** alta affidabilità e ripristino rapido (copia semplice).
- **RAID 5:** scelta comune per grandi quantità di dati; buon compromesso tra spazio, prestazioni e protezione.
- **RAID 6:** usato in *storage array* dove la doppia protezione è fondamentale (tolleranza al guasto di due dischi).

Problemi connessi a RAID

Il RAID protegge solo dagli errori derivanti dai supporti fisici (guasti del disco). Non protegge da:

- Errori software (puntatori a file errati, scrittura accidentale).
- Operazioni di scrittura incomplete non gestite correttamente.
- **ZFS (Solaris): Soluzione all'integrità:** il file system ZFS usa un approccio innovativo:
 - **Checksum Ereditata:** calcola un checksum per ogni blocco (dati e metadati) e la memorizza nel puntatore che punta a quel blocco (es. un i-node contiene la checksum dei suoi blocchi dati).
 - **Auto-riparazione:** se il mirroring è attivo e una checksum non corrisponde, ZFS sostituisce automaticamente il blocco corrotto con la copia valida. Questo garantisce un livello superiore di coerenza.
- **Flessibilità (ZFS):** ZFS combina la gestione dei file system e dei volumi in un unico **Storage Pool**. Tutta l'area libera del pool è disponibile per tutti i file system in esso contenuti, eliminando i limiti artificiali di dimensione e la necessità di ridimensionamento manuale.

Object Storage (archiviazione a oggetti)

L'object storage è un approccio di archiviazione orientato al **computer/programma**, diverso dai file system tradizionali orientati all'utente.

- **Struttura:** i dati sono inseriti in un **pool di archiviazione** come **oggetti**, ciascuno identificato da un **ID univoco**. Non fornisce una struttura di directory navigabile.
- **Operazioni tipiche:** creare un oggetto e ricevere l'ID, accedere all'oggetto tramite l'ID, eliminare l'oggetto tramite l'ID.
- **Protezione dati:** gestita dal software di Object Storage (es. Hadoop HDFS, Ceph). Spesso si basa sulla memorizzazione di N copie dell'oggetto su N computer diversi (replica software) anziché su array RAID hardware.
- **Vantaggi:**
 - **Costo inferiore:** può utilizzare HW di base.
 - **Scalabilità orizzontale:** la capacità massima è aumentata semplicemente aggiungendo più computer al pool (fino a dimensioni Petabyte).
 - **Dati autodescrittivi:** ogni oggetto include una descrizione dei suoi contenuti (content-addressable storage).

- **Utilizzo:** ideale per la memorizzazione di **grandi quantità di dati** non strutturati (Big Data), non per accesso casuale ad alta velocità (es. contenuti di Google Search, Dropbox, Spotify, ecc...).